

**SIMATS ENGINEERING
ASSESSMENT TOOL 2**

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.(BL3)

Assessment Tool Weightage: 35%

QUESTIONS: 3

**Duration : 1 Hour
Total Marks:30**

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

*I ___P.Lakshmi Gayathri Navaneetha___ (192321169) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

P.Gayathri

Signature of the student:

Annexure A

Constraint-Based Problem Solving

1. A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) such that:

Constraints:

- i. D1 cannot work Night shift
- ii. D2 must work before D3 (Morning < Afternoon < Night)
- iii. Only one doctor per shift
- iv. D3 cannot work Morning
- v. All doctors must be assigned exactly one shift

Apply Backtracking Search to find a valid assignment with all intermediate steps and constraint checks. State the final valid schedule

1. Problem Understanding

Three doctors (D1, D2, D3) must be assigned to Morning, Afternoon, and Night shifts. The assignment must satisfy these constraints:

- D1 cannot work the Night shift.
- D2 must work before D3 (D2's shift must precede D3's shift).
- Only one doctor can be assigned to each shift.
- D3 cannot work the Morning shift.
- Every doctor must be assigned exactly one shift.

2. Backtracking Search: Step-by-Step

Backtracking assigns one shift at a time and checks the constraints after each assignment. If an assignment violates a constraint, it is rejected and another shift is tried.

Step	Assignment	Constraint Check	Result
1	D1 → Morning	D1 is not assigned to Night.	Valid
2	D2 → Afternoon	D2 works before D3 if D3 is assigned Night.	Valid so far
3	D3 → Night	D3 is not assigned to Morning.	Valid
4	Check all shifts	Each shift has exactly one doctor.	Valid
5	Check all doctors	Every doctor has exactly one shift.	Solution found

Backtracking Pseudocode

```
BACKTRACK(doctor):
    if all doctors are assigned:
        return solution

    for each available shift:
        assign shift to doctor

        if all constraints are satisfied:
            if BACKTRACK(next doctor) succeeds:
                return solution

        remove assignment

    return failure
```

3. Final Valid Schedule

Shift	Doctor
Morning	D1
Afternoon	D2
Night	D3

Conclusion: Backtracking Search finds a valid assignment by checking constraints at every step. The final schedule satisfies all the stated conditions without conflicts.

2. A robot operates in a grid environment (5×5). It must move from Start (S) to Goal (G). Obstacles block certain paths.

Constraints:

- i. **Movement allowed: Up, Down, Left, Right**
- ii. **Cost of each move = 1**
- iii. **Cannot pass through obstacles**
- iv. **Use Manhattan Distance heuristic**

Using above constraints and BFS Search Algorithm Show step-by-step node expansion and priority queue updates and determine the optimal path and cost.

1. Problem Understanding

A robot must move from Start (S) to Goal (G) in a 5×5 grid. Obstacles block certain cells. Movement is allowed Up, Down, Left, and Right; every move costs 1; obstacles cannot be crossed; Manhattan Distance is the specified heuristic.

The question does not specify the obstacle locations. For the worked example below, the following illustrative grid and coordinates are assumed. Coordinates are (row, column), beginning at (1,1).

	Column 1	Column 2	Column 3	Column 4	Column 5
Row 1	S	.	.	#	.
Row 2	.	.	.	#	.
Row 3	.	.	#	.	.
Row 4	.	.	.	.	.
Row 5	#	#	.	.	G

S = Start; G = Goal; # = obstacle; . = free cell. Start = (1,1), Goal = (5,5). Obstacles = (1,4), (2,4), (3,3), (5,1), and (5,2).

2. Manhattan Distance Heuristic

Manhattan Distance estimates the number of horizontal and vertical moves to the goal, ignoring obstacles:

$$h(n) = |\text{row}(n) - \text{row}(G)| + |\text{column}(n) - \text{column}(G)|$$

At the start, $h(S) = |1 - 5| + |1 - 5| = 8$.

BFS does not use a heuristic; it explores the grid level by level. Manhattan Distance is typically used by A* Search, where $f(n) = g(n) + h(n)$. BFS is appropriate here because every move has equal cost.

3. BFS Node Expansion and Queue Updates

BFS uses a first-in, first-out (FIFO) queue. The following trace uses the assumed grid and a fixed neighbor order. Already visited cells are not added again.

Step	Expanded Node	Newly Discovered Nodes	Queue After Expansion
1	(1,1)	(1,2), (2,1)	(1,2), (2,1)
2	(1,2)	(1,3), (2,2)	(2,1), (1,3), (2,2)
3	(2,1)	(3,1)	(1,3), (2,2), (3,1)
4	(1,3)	(2,3)	(2,2), (3,1), (2,3)
5	(2,2)	None (neighbors visited)	(3,1), (2,3)
6	(3,1)	(3,2), (4,1)	(2,3), (3,2), (4,1)
7	(2,3)	None (blocked/visited)	(3,2), (4,1)
8	(3,2)	(4,2)	(4,1), (4,2)
9	(4,1)	None (neighbors visited)	(4,2)
10	(4,2)	(4,3)	(4,3)
11	(4,3)	(4,4), (3,4)	(4,4), (3,4)
12	(4,4)	(4,5), (5,4)	(3,4), (4,5), (5,4)
13	(3,4)	(3,5)	(4,5), (5,4), (3,5)
14	(4,5)	(5,5)	(5,4), (3,5), (5,5)
15	(5,5)	Goal reached	Stop

4. Shortest Path and Cost

One shortest path in the assumed grid is:

$(1,1) \rightarrow (1,2) \rightarrow (1,3) \rightarrow (2,3) \rightarrow (2,2) \rightarrow (2,1) \rightarrow (3,1) \rightarrow (3,2) \rightarrow (4,2) \rightarrow (4,3) \rightarrow (4,4) \rightarrow (4,5) \rightarrow (5,5)$

The route contains 12 moves, so total cost = $12 \times 1 = 12$.

Conclusion: BFS finds a shortest path in an unweighted grid because each move has equal cost. The exact path and expansion order depend on the actual obstacle locations and the order in which neighbors are examined.

3. An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The robot operates in a partially observable environment where some paths are blocked, and it must make decisions with limited information. The building is represented as a grid of rooms. The robot starts at position S and must reach a survivor located at G.

Constraints:

- The robot can move up, down, left, right
- Some cells are blocked (obstacles) and cannot be traversed
- The robot has limited sensing capability (can only observe adjacent cells)
- Each movement has a cost of 1, but entering risky zones has an additional cost of 2
- The robot must minimize total cost while ensuring safe navigation
- The robot must update its knowledge dynamically (online search scenario)

(a) Analyze the Constraints

Constraint	Effect on Decision-Making
Four-direction movement	The robot can move only Up, Down, Left, or Right.
Blocked cells	Obstacles must be avoided; an alternative route is required.
Limited sensing	The complete grid is not known initially; only adjacent cells can be observed.
Normal movement cost	Each ordinary movement adds 1 to the path cost.
Risky-zone cost	A risky cell costs $1 + 2 = 3$ units to enter.
Dynamic knowledge	The robot updates its map after sensing new cells.
Cost minimization and safety	It should avoid obstacles and select a safe feasible route with the lowest total cost.

The total cost of a route is $C = \sum(1 + R_i)$, where $R_i = 2$ for a risky cell and $R_i = 0$ for a normal cell. Thus, a normal move costs 1 and a risky-cell entry costs 3.

(b) Solution Using Uniform Cost Search and an Online Search Agent

Uniform Cost Search (UCS) expands the frontier node with the smallest accumulated path cost. It is suitable because different cells can have different entry costs. An online search agent complements UCS by sensing the environment and updating its map while moving.

Step 1: Initialize: Set the start cost $g(S) = 0$. Add the start cell to a priority queue. Mark cells whose status is unknown as unexplored.

Step 2: Sense adjacent cells: Observe the cells immediately above, below, left, and right where applicable. Classify them as free, blocked, risky, or unknown.

Step 3: Update costs and queue: For each reachable, non-blocked neighbor v , calculate $g(v) = g(u) + \text{entry_cost}(v)$. Add 1 for a normal cell and 3 for a risky cell.

Step 4: Expand the lowest-cost node: Remove the lowest-cost node from the priority queue, sense its surroundings, and update the map. Store the predecessor for each best-known route.

Step 5: Replan as needed: If new information reveals an obstacle or risk, update the map and recompute the route using the currently known graph.

Step 6: Stop at the goal: When G is reached, reconstruct the route from stored predecessors and calculate the total cost.

Uniform Cost Search Pseudocode

```
UCS_Online(S, G):
    priority_queue = [(0, S)]
    cost[S] = 0

    while priority_queue is not empty:
        remove node u with minimum cost

        if u == G:
            return path and cost

        sense adjacent cells
        update map with new information

        for each neighbor v of u:
            if v is not blocked:
                new_cost = cost[u] + entry_cost(v)

                if v is new or new_cost < cost[v]:
                    cost[v] = new_cost
                    parent[v] = u
                    insert or update v in priority_queue

    return failure if no route exists
```

Note: In a partially observable environment, the robot cannot assume unknown cells are free. It must sense, update its map, and replan. Standard UCS guarantees a minimum-cost path on the graph it is given, assuming non-negative edge costs.

(c) Application of AI Concepts

1. Intelligent Agents

An intelligent agent perceives its environment through sensors and acts through actuators. The robot's sensors detect nearby obstacles and risky cells; its agent program chooses a move; its actuators execute the move. It repeatedly perceives, decides, and acts to reach the survivor.

2. Rationality

A rational agent selects the action expected to achieve the best result using its available information. The robot avoids known obstacles, considers risky-zone costs, chooses a lower-cost feasible route, and replans when new information changes the situation. Rationality does not mean it knows the perfect path in advance.

3. Environment Type

Property	Classification	Reason
Observability	Partially observable	The robot senses only adjacent cells.
Determinism	Mostly deterministic	Movement outcomes are predictable under the stated rules.
Structure	Sequential	Each move affects later decisions.
Dynamics	Dynamic or initially unknown	The map may change or be discovered gradually.
State space	Discrete	The robot occupies individual grid cells.
Agents	Single-agent	One robot makes navigation decisions.
Cost structure	Non-uniform	Risky cells cost more than normal cells.

4. Knowledge Representation

The robot maintains an internal map of known free cells, obstacles, risky cells, unknown cells, its current position, accumulated cost, and the best-known route to each discovered cell.

(d) Justification for the Chosen Approach

- Cost optimization: UCS expands the lowest-cost path first.
- Obstacle avoidance: Known blocked cells are excluded from expansion.
- Risk awareness: Risky-zone costs are included in path calculations.
- Optimality: With non-negative costs and a known graph, UCS finds a minimum-cost path to the goal.
- Online adaptation: Sensing and replanning allow the robot to respond when it discovers new information.

BFS is not the best choice for the rescue scenario because it assumes equal step costs. UCS is more suitable for unequal costs, but ordinary UCS assumes the graph is known. Combining

UCS with online sensing and replanning addresses the partial observability. If safety is a strict requirement, risky cells should also be managed by an explicit safety policy rather than by cost alone.

Conclusion: Uniform Cost Search combined with online sensing and replanning is suitable for rescue navigation because it accounts for unequal movement costs, avoids known obstacles, incorporates newly discovered information, and finds a minimum-cost route on the currently known graph.

Final Summary

Question	AI Technique	Main Result
1	Backtracking Search	Valid doctor-shift assignment
2	Breadth-First Search (BFS)	Shortest path for equal movement costs
3	Uniform Cost Search + Online Agent	Minimum-cost navigation with dynamic knowledge

Important note: For Question 2, the obstacle positions were not supplied in the question. The grid and path above are an illustrative example. If a specific grid was provided by the instructor, replace the assumed obstacles and recalculate the node expansions and route cost.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided